AG Technologies

Service Recherche

Cahier des Charges Technique et Fonctionnel

Version 1.2 — 21 mars 2026

Statut : **Implementation-ready**

Attribut	**Valeur**
Document	CDC Technique et Fonctionnel
Service	Search Service
Version	1.2
Classification	Confidentiel — Usage interne
Stack	Django / DRF / Elasticsearch / Redis / PostgreSQL
Dépendances validées	Auth v2.1, Users v1.0, Notification v1.2, Média v1.4

AG Technologies — Document confidentiel à usage interne exclusivement.

Historique des Révisions

Version	Date	Description	Auteur
1.0	21 mars 2026	Création initiale du document	Équipe Ingénierie
1.1	21 mars 2026	Post-audit : alignement `platform_id` UUID (Auth v2.1). Préfixe `/api/v1` sur tous les endpoints. Contrainte UNIQUE `(name, platform_id)`. Table `popular_searches` documentée. `TIMESTAMPTZ` partout. Contrats inter-services formalisés (Auth, RGPD). Codes HTTP sur tous les endpoints. Identité `user_id = users_auth.id`. Isolation multi-tenant documentée. Celery ajouté à la stack. Contrat d'événements d'indexation. Garde-fous anti-abus. Health check détaillé. PlantUML en annexe.	Équipe Architecture
1.2	21 mars 2026	Version finale : règle d'architecture Search $\neq$ Users (profils jamais indexés par Search). Gouvernance des index (nommage, quotas, bulk max, suppression prod). Matrice d'autorisation exhaustive par endpoint (JWT / S2S / Admin / Public). Statut passé à *Implementation-ready*.	Équipe Architecture

Table des matières

1 Introduction **5**
1.1 Objet du Document . . . 5
1.2 Destinataires . . . 5
1.3 Périmètre du Service . . . 5
1.4 Conventions écosystème AG Technologies . . . 6
1.5 Règle d'architecture : frontière Search / Users (v1.2) . . . 6

2 Contexte & Problématique **6**
2.1 Pourquoi Ce Service Existe . . . 6
2.2 Problèmes Résolus . . . 7
2.3 Positionnement dans l'Architecture Globale . . . 7

3 Vision & Objectifs **7**
3.1 Concept du Service . . . 7
3.2 Objectifs MVP & Long Terme . . . 8
3.3 Indicateurs de Performance (KPIs) . . . 8

4 Parties Prenantes & Utilisateurs **8**
4.1 Équipe Technique . . . 8
4.2 Services Consommateurs . . . 8

5 Besoins Fonctionnels **9**
5.1 Module Index Dynamiques . . . 9
5.1.1 Types de Champs Supportés . . . 9
5.1.2 Propriétés Configurables par Champ . . . 9
5.1.3 Exemple de Schéma — Index Produits (AGT-Market) . . . 9
5.2 Module Indexation . . . 10
5.3 Module Recherche Full-Text . . . 10
5.4 Module Auto-Complétion . . . 10
5.5 Module Filtres & Facettes . . . 10
5.6 Module Recherche par Similarité . . . 10
5.7 Module Historique & Statistiques . . . 11

6 Besoins Non-Fonctionnels **11**
6.1 Performance . . . 11
6.2 Sécurité . . . 11
6.2.1 Garde-fous anti-abus (v1.1) . . . 11
6.2.2 Gouvernance des index (v1.2) . . . 12
6.3 Scalabilité . . . 12
6.4 Disponibilité . . . 12

7 Architecture Technique **13**
7.1 Stack Technique . . . 13
7.2 Schéma d'Architecture . . . 13
7.3 Intégrations Externes . . . 14

8 Modélisation **14**
8.1 Diagramme de Classes . . . 14
8.2 Modèle Logique de Données (MLD) . . . 15
8.2.1 Table `indexes_registry` . . . 15
8.2.2 Table `index_schemas` . . . 15
8.2.3 Table `search_history` . . . 16
8.2.4 Table `search_configs` . . . 16
8.2.5 Table `synonyms` . . . 17
8.2.6 Table `popular_searches` (v1.1 — documentée) . . . 17

9 Contrats d'API **17**
9.1 Matrice d'autorisation par endpoint (v1.2) . . . 17
9.2 Health Check . . . 18
9.2.1 `GET /api/v1/search/health` . . . 18
9.3 Gestion des Index . . . 19
9.3.1 `POST /api/v1/search/indexes` . . . 19
9.3.2 `GET /api/v1/search/indexes` . . . 19
9.3.3 `GET /api/v1/search/indexes/{indexName}` . . . 19
9.3.4 `PUT /api/v1/search/indexes/{indexName}/schema` . . . 19
9.3.5 `DELETE /api/v1/search/indexes/{indexName}` . . . 19
9.3.6 `POST /api/v1/search/indexes/{indexName}/reindex` . . . 19
9.4 Indexation . . . 20
9.4.1 `POST /api/v1/search/indexes/{indexName}/documents` . . . 20
9.4.2 `PUT /api/v1/search/indexes/{indexName}/documents/{docId}` . . . 20
9.4.3 `DELETE /api/v1/search/indexes/{indexName}/documents/{docId}` . . . 20
9.4.4 `POST /api/v1/search/indexes/{indexName}/documents/bulk` . . . 20
9.5 Recherche . . . 21
9.5.1 `POST /api/v1/search/query` . . . 21
9.5.2 `GET /api/v1/search/autocomplete` . . . 22
9.6 Historique de Recherche . . . 22
9.6.1 `GET /api/v1/search/history` . . . 22
9.6.2 `DELETE /api/v1/search/history` . . . 22
9.6.3 `DELETE /api/v1/search/history/by-user/{userId}` (v1.1 — S2S) . . . 22
9.6.4 `GET /api/v1/search/popular` . . . 22
9.6.5 `GET /api/v1/search/no-results` . . . 23
9.7 Configuration . . . 23
9.7.1 `GET /api/v1/search/indexes/{indexName}/config` . . . 23
9.7.2 `PUT /api/v1/search/indexes/{indexName}/config` . . . 23
9.7.3 `PUT /api/v1/search/indexes/{indexName}/synonyms` . . . 23
9.8 Statistiques . . . 23
9.8.1 `GET /api/v1/search/stats` . . . 23
9.8.2 `GET /api/v1/search/stats/terms` . . . 24

10 Contrats Inter-Services **24**
10.1 Recherche → Auth (JWT utilisateur) . . . 24
10.2 Recherche → Auth (token S2S) . . . 24
10.3 Users → Recherche (Purge RGPD) . . . 24
10.4 Contrat d'événements d'indexation . . . 25
10.4.1 Format de message standard . . . 25
10.4.2 Règles de traitement . . . 25

11 Infrastructure **26**
11.1 Composants d'Infrastructure 26
11.2 Variables d'Environnement 26
11.3 Docker Compose (Extrait) 26

12 Recommandations & Évolutions **28**
12.1 Réindexation sans Downtime (Alias Elasticsearch) 28
12.2 Recherche Sémantique par Embeddings 28
12.3 Synonymes Auto-Enrichis 28
12.4 Monitoring Elasticsearch 28
12.5 Suppression : Stratégies 28

A Annexe A — Diagramme de Classes (PlantUML) **29**

B Annexe B — Glossaire **32**

1 Introduction

1.1 Objet du Document

Ce document constitue le Cahier des Charges Technique et Fonctionnel du Service Recherche (Search Service) d'AG Technologies. Il définit de manière exhaustive les objectifs, les fonctionnalités, l'architecture technique et les contrats d'API de ce service partagé.

Il sert de référence officielle pour :

- La conception et la modélisation du service
- Le développement et l'implémentation
- Les décisions techniques et les arbitrages d'architecture
- L'intégration par les équipes et services consommateurs

1.2 Destinataires

Destinataire	Rôle
Équipe Backend	Développement et maintenance du service
Équipe Frontend	Intégration des APIs de recherche et d'auto-complétion
Équipes consommatrices	AGT-Market, AGT-Bot, SALMA, Service Chatbot, et toute future plateforme AG Technologies
Lead Technique / Architecte	Validation des choix techniques
Chef de Projet	Suivi et validation fonctionnelle

1.3 Périmètre du Service

Périmètre inclus

Gestion des index dynamiques, indexation unitaire et bulk, recherche full-text avec scoring, auto-complétion, filtres avancés avec facettes, boost de résultats, recherche par similarité, historique de recherche, statistiques et configuration par index.

Attention

Hors périmètre : authentification (déléguée au Service Auth v2.1), stockage des données source (chaque plateforme conserve sa propre base), facturation du boost (déléguée au Service Abonnement), accès direct à Elasticsearch (jamais exposé aux plateformes), gestion des rôles et permissions (déléguée au Service Users v1.0).

1.4 Conventions écosystème AG Technologies

Note

Ce service s'intègre dans un écosystème dont les conventions suivantes sont **non négociables** (validées dans Auth v2.1, Users v1.0, Notification v1.2, Média v1.4) :

- `platform_id` = UUID du registre Auth (`platforms.id`)
- Identité utilisateur = `users_auth.id` (= `sub` du JWT)
- JWT validé via `GET /auth/verify-token` ; token S2S via `POST /auth/s2s/introspect`
- Préfixe API : `/api/v1`
- Rôles et permissions gérés par Users, pas par Auth
- RGPD orchestré par Users
- Stack : Django/DRF pour Auth, Users, Notif ; Node.js/NestJS pour Média ; Django/DRF pour Recherche
- Timestamps : `TIMESTAMPTZ` (avec timezone)

1.5 Règle d'architecture : frontière Search / Users (v1.2)

Attention

Règle non négociable : Le Service Recherche n'indexe **jamais** de profils utilisateurs. La recherche d'utilisateurs (par nom, email, téléphone) est une responsabilité exclusive du Service Users (endpoint interne Users, cible < 500ms).
Le Service Recherche indexe uniquement des **données métier** fournies par les plateformes consommatrices : produits (AGT-Market), FAQ (AGT-Bot), formations (SALMA), base de connaissances (Chatbot).
Si une plateforme a besoin de rechercher des utilisateurs *et* des produits, elle appelle deux services distincts :

- `GET /api/v1/users?search=...` → Service Users
- `POST /api/v1/search/query` → Service Recherche

Cette séparation garantit que les données PII (email, téléphone, date de naissance) ne transitent jamais dans Elasticsearch, conformément à la stratégie RGPD de l'écosystème.

2 Contexte & Problématique

2.1 Pourquoi Ce Service Existe

Dans un écosystème multi-plateformes, chaque application a besoin de capacités de recherche performantes : produits sur AGT-Market, FAQ sur AGT-Bot, formations sur SALMA. Sans service centralisé, chaque plateforme développerait sa propre intégration Elasticsearch, entraînant :

- **Duplication de la complexité** — chaque équipe gère ses propres index, mappings et analyseurs
- **Incohérence de l'expérience** — résultats de qualité hétérogène selon la plateforme
- **Coûts d'infrastructure multipliés** — plusieurs clusters Elasticsearch au lieu d'un seul
- **Exposition directe d'Elasticsearch** — risques de sécurité et de mauvaise utilisation
- **Absence de statistiques unifiées** — impossible de mesurer la qualité globalement

2.2 Problèmes Résolus

Problème	Solution apportée
Intégration ES complexe	API REST simplifiée masquant toute la complexité Elasticsearch
Index rigides codés en dur	Index 100% dynamiques définis par les plateformes via l'API
Recherche lente et peu pertinente	Full-text avec scoring, boost, fuzzy et analyseurs configurables
Pas de suggestions temps réel	Auto-complétion dédiée optimisée $<$ 50ms
Statistiques absentes	Historique, termes populaires, recherches sans résultat

2.3 Positionnement dans l'Architecture Globale

Le Service Recherche est un service de base consommé par toutes les plateformes. Il encapsule Elasticsearch et n'expose que son API REST.

```
  AGT-Market    AGT-Bot    SALMA    Chatbot
       |            |         |        |
       +------+-----+----+----+--------+
              |           |
         Search Service (API REST)
              |           |
    +---------+-----------+----------+
    |         |           |          |
Service Auth  ES       Redis    PostgreSQL
```

3 Vision & Objectifs

3.1 Concept du Service

Le Service Recherche est le moteur de recherche unifié et générique de toutes les plateformes AG Technologies. Il ne connaît aucun concept métier. Il gère des index abstraits contenant des documents avec des champs définis dynamiquement par les plateformes.

Principe fondamental

Elasticsearch est entièrement encapsulé. Les plateformes n'y accèdent jamais directement — tout passe par l'API du Service Recherche. La plateforme crée ses index, définit ses schémas et indexe ses documents. Le service gère la complexité.

3.2 Objectifs MVP & Long Terme

Phase	Objectifs
MVP	Index dynamiques avec schéma configurable ; recherche full-text multi-champs avec scoring ; filtres avancés et facettes ; auto-complétion (< 50ms) ; fuzzy matching ; boost de résultats ; indexation unitaire et bulk ; historique de recherche ; API REST complète
Long terme	Recherche sémantique par embeddings vectoriels ; personnalisation des résultats par utilisateur ; synonymes auto-enrichis basés sur les recherches ; A/B testing des algorithmes de ranking ; monitoring Elasticsearch avancé

3.3 Indicateurs de Performance (KPIs)

KPI	Cible MVP	Cible Long Terme
Temps de réponse recherche full-text	< 200ms	< 100ms
Temps auto-complétion	< 50ms	< 30ms
Indexation unitaire	< 100ms	< 50ms
Indexation bulk (1000 docs)	< 5s	< 2s
Disponibilité du service	≥ 99%	≥ 99,5%
Taux d'erreur API	< 1%	< 0,5%

4 Parties Prenantes & Utilisateurs

4.1 Équipe Technique

Rôle	Responsabilités
Ingénieur Backend	Développement API, intégration Elasticsearch, pipeline d'indexation
Lead Technique	Architecture, tuning Elasticsearch, décisions de performance
Ingénieur DevOps	Cluster ES, Redis, monitoring, volumes persistants
Ingénieur Frontend	Intégration barre de recherche, auto-complétion, facettes

4.2 Services Consommateurs

Service	Usage
AGT-Market	Index produits et boutiques ; filtres prix/catégorie ; boost sponsorisé ; auto-complétion
AGT-Bot	Index FAQ ; recherche par similarité pour trouver la réponse la plus proche
SALMA	Index formations ; recherche par niveau, prix, domaine
Service Chatbot	Recherche sémantique dans la base de connaissances
Futures plateformes	*Tout nouveau service AG Technologies peut créer ses propres index dynamiquement via l'API. La liste ci-dessus n'est pas figée.*

5 Besoins Fonctionnels

5.1 Module Index Dynamiques

Principe clé

Aucun index n'est codé en dur. Chaque plateforme crée et gère ses propres index via l'API, en définissant librement le schéma de ses documents.

5.1.1 Types de Champs Supportés

Type	Description & usage
`text`	Contenu textuel recherchable en full-text (analysé, tokenizé)
`keyword`	Valeur exacte, filtrable et triable (catégorie, statut, ID)
`number`	Valeur numérique, filtrable et triable (prix, note, quantité)
`date`	Date/heure, filtrable et triable
`boolean`	Vrai/faux, filtrable
`geo_point`	Coordonnées géographiques — recherche par distance (futur)

5.1.2 Propriétés Configurables par Champ

Propriété	Description
`searchable`	Le champ est inclus dans la recherche full-text
`filterable`	Le champ peut être utilisé comme filtre
`sortable`	Le champ peut servir de critère de tri
`autocomplete`	Le champ est utilisé pour l'auto-complétion
`boost_weight`	Poids du champ dans le score de pertinence

5.1.3 Exemple de Schéma — Index Produits (AGT-Market)

Listing 1 – Exemple de mapping index products

```
{
  "index": "products",
  "platform_id": "uuid-agt-market-123",
  "analyzer": "french",
  "fields": {
    "title": { "type": "text", "searchable": true,
      "autocomplete": true, "boost_weight": 3 },
    "description": { "type": "text", "searchable": true,
      "boost_weight": 1 },
    "price": { "type": "number", "filterable": true,
      "sortable": true },
    "category": { "type": "keyword", "filterable": true },
    "rating": { "type": "number", "filterable": true,
      "sortable": true },
    "is_boosted": { "type": "boolean", "filterable": true },
    "created_at": { "type": "date", "filterable": true,
      "sortable": true }
  }
}
```

5.2 Module Indexation

- Indexation unitaire : ajout ou mise à jour d'un document dans un index (< 100ms)
- Suppression unitaire : retrait d'un document de l'index
- Indexation bulk : ajout, mise à jour et suppression de centaines de documents en une requête
- Réindexation complète : reconstruction de l'index après modification du schéma (via alias ES, cf. section 9.3.6)
- Les plateformes sont responsables de maintenir leurs index synchronisés avec leur base de données
- Pattern recommandé : événement asynchrone via message broker après chaque modification en base (cf. section 10.4)

5.3 Module Recherche Full-Text

- Recherche multi-champs dans tous les champs marqués `searchable`
- Analyse linguistique : tokenisation, stemming, suppression des stop words
- Support français et anglais (analyseurs configurables par index)
- Fuzzy matching : tolérance aux fautes de frappe configurable
- Score de pertinence : chaque résultat est classé selon sa correspondance
- Boost : les documents avec `is_boosted = true` remontent dans les résultats
- Pagination (`page`, `limit`) et tri multi-critères

5.4 Module Auto-Complétion

- Endpoint dédié optimisé pour la vitesse (< 50ms)
- Suggestions basées sur les champs marqués `autocomplete`
- Suggestions enrichies par les recherches populaires (historique agrégé anonymisé)
- Nombre de suggestions configurable (défaut : 8)
- Support du prefix matching : `"chaus"` → `"chaussures"`, `"chaussettes"`
- Highlight : la partie matchée est identifiée dans la suggestion

5.5 Module Filtres & Facettes

- Filtre exact : `category = "Électronique"`
- Filtre range : `price >= 5000 AND price <= 50000`
- Filtre multi-valeurs : `category IN ["Électronique", "Mode"]`
- Filtre booléen : `is_available = true`
- Filtre date : `created_at >= "2026-01-01"`
- Filtre géographique (futur) : distance ≤ 10km de (lat, lng)
- Les filtres sont combinables entre eux (AND logique)
- Facettes : le service retourne les compteurs par valeur de filtre pour alimenter l'UI

5.6 Module Recherche par Similarité

- Comprend le sens de la requête au-delà des mots exacts
- `"téléphone portable"` trouve aussi `"smartphone"`, `"mobile"`
- Utile pour le Service Chatbot (FAQ la plus proche d'une question reformulée)
- MVP : synonymes + fuzzy matching Elasticsearch
- Futur : embeddings vectoriels via API IA stockés dans Elasticsearch `dense_vector`

5.7 Module Historique & Statistiques

- Chaque recherche est enregistrée : requête, filtres, index, `user_id`, timestamp, nb résultats
- L'utilisateur peut consulter et supprimer son historique (RGPD)
- Les recherches populaires sont agrégées de façon anonymisée (table `popular_searches`)
- Statistiques admin : termes les plus recherchés, recherches sans résultat, temps de réponse

6 Besoins Non-Fonctionnels

6.1 Performance

Exigence	Valeur cible
Temps de réponse recherche full-text	$<$ 200ms
Temps de réponse auto-complétion	$<$ 50ms
Indexation unitaire	$<$ 100ms
Indexation bulk (1000 documents)	$<$ 5 secondes
Cache Redis (auto-complétions fréquentes)	TTL configurable
RAM Elasticsearch (heap JVM)	$\geq$ 2 GB dédiés

6.2 Sécurité

- **Authentification obligatoire** : tous les endpoints (sauf health check) requièrent un token Bearer valide (Service Auth v2.1)
- **Isolation des index** : une plateforme ne peut accéder qu'à ses propres index. Le `platform_id` est extrait du JWT vérifié (champ `platform_id` du payload, cf. Auth v2.1 section 7.4). Toute requête est automatiquement scopée côté serveur — jamais de trust client
- **Elasticsearch jamais exposé** : aucun endpoint ES n'est accessible depuis l'extérieur
- **Données historique RGPD** : droit à la suppression de l'historique par utilisateur
- **Rate limiting** : protection contre les abus via Redis

6.2.1 Garde-fous anti-abus (v1.1)

Protection	Description	Valeur
`limit` max	Plafond de résultats par requête	100
Timeout query ES	Timeout côté Elasticsearch pour éviter les requêtes lourdes	5s
Wildcards	Interdiction des wildcards en début de terme (`*chaussure`)	Bloqué
Fuzzy distance max	Distance d'édition maximale autorisée	2
Rate limit recherche	Requêtes de recherche par minute par plateforme	600/min
Rate limit indexation	Requêtes d'indexation par minute par plateforme	120/min

6.2.2 Gouvernance des index (v1.2)

Règle	Description
Convention de nommage	`snake_case`, lettres minuscules, pas d'espaces ni de caractères spéciaux. Exemples valides : `products`, `faq_entries`, `formations_salma`. Validé par regex côté serveur : `^[a-z][a-z0-9_]{2,49}$`
Quota d'index par plateforme	Maximum 20 index actifs par plateforme (configurable via variable d'environnement `MAX_INDEXES_PER_PLATFORM`). Au-delà → erreur 429.
Taille max bulk payload	Maximum 500 opérations par requête bulk. Maximum 10 MB par requête. Au-delà → erreur 413 Payload Too Large.
Taille max document	Un document individuel ne peut pas dépasser 1 MB (données JSON).
Suppression d'index en prod	`DELETE /api/v1/search/indexes/{indexName}` est irréversible. Le statut passe à `deleted` en PostgreSQL. L'alias ES et l'index physique sont supprimés. Aucune récupération possible. En production, cette opération devrait être protégée par une permission spécifique côté plateforme (vérifiée via Service Users).
Nombre max de champs par index	Maximum 100 champs par schéma d'index. Protection contre l'explosion du mapping ES.

6.3 Scalabilité

- Scalabilité horizontale de l'API : plusieurs instances derrière un load balancer
- Elasticsearch scalable par ajout de nœuds (shards et replicas configurables)
- Cache Redis pour les auto-complétions et recherches populaires fréquentes
- Cohérence éventuelle acceptable : l'index reflète l'état de la base source avec un léger délai
- Le service est stateless — tout l'état est dans Elasticsearch et Redis

6.4 Disponibilité

- Disponibilité cible : $\geq$ 99,5%
- Elasticsearch sur volume Docker persistant (SSD recommandé)
- Health check exposé sur `GET /api/v1/search/health` (état API + état cluster ES + Redis + PostgreSQL)
- Déploiement sans interruption (rolling updates)

7 Architecture Technique

7.1 Stack Technique

Composant	Technologie	Rôle
Runtime	Django (Python 3.11+)	Logique applicative, orchestration des requêtes ES
Framework API	Django REST Framework	Exposition des endpoints REST
Moteur de recherche	Elasticsearch 8.x	Stockage, indexation, recherche full-text
Client ES	`elasticsearch-py`	Librairie Python officielle pour Elasticsearch
Base de données	PostgreSQL 15+	Métadonnées, historique, synonymes, configs
Cache	Redis 7+	Auto-complétions, recherches populaires, rate limiting
Workers async	Celery	Consommation des événements d'indexation
Message Broker	RabbitMQ	File d'attente pour indexation asynchrone
Conteneurisation	Docker + Docker Compose	Elasticsearch + API dans des conteneurs dédiés

Note

Choix de stack (v1.1) : Django/DRF est retenu pour cohérence avec Auth, Users et Notification. Celery + RabbitMQ est ajouté pour le pipeline d'indexation asynchrone, conformément au pattern utilisé dans le Service Notification (Celery + RabbitMQ pour le traitement des envois).

7.2 Schéma d'Architecture

```
Couche clients
+-------------+----------+---------+----------+
| AGT-Market  | AGT-Bot  | SALMA   | Chatbot  |
+------+------+----+-----+----+----+-----+----+
       |           |          |           |
Couche securite
+------+-----------+----------+-----------+----+
|    Auth Middleware -> Service Auth (JWT)      |
+------+-----------+----------+-----------+----+
       |           |          |           |
Couche application
+------+-----------+----------+-----------+----+
|        API REST -- Search Service            |
+------+-----------+----------+-----------+----+
|  Recherche | Autocompl. | Indexation | Histo. |
+------+-----+------+-----+-----+------+------+
       |            |           |
Couche persistance
```

```
+------+-----+ +----+----+ +--+--------+
| PostgreSQL | | Redis   | | Celery    |
+------------+ +---------+ +-----+-----+
                                 |
                            +-----+-----+
                            | RabbitMQ  |
                            +-----------+
       |
+------+----------+
| Elasticsearch   |
+-----------------+
```

7.3 Intégrations Externes

Service externe	Rôle	Phase
Service Auth v2.1	Validation obligatoire du token Bearer sur chaque requête (`GET /auth/verify-token`)	MVP
Elasticsearch 8.x	Moteur de recherche et stockage des index	MVP
Redis 7+	Cache auto-complétion, recherches populaires, rate limiting	MVP
RabbitMQ	Indexation asynchrone depuis les plateformes (via Celery)	MVP
PostgreSQL 15+	Métadonnées d'index, historique, synonymes, configs	MVP
API Embeddings (OpenAI)	Génération de vecteurs pour recherche sémantique	Futur

8 Modélisation

Note

Note d'architecture : Les données recherchables (documents indexés) résident dans **Elasticsearch**, pas en PostgreSQL. PostgreSQL stocke uniquement les métadonnées de gestion : registre des index, configurations, historique de recherche, recherches populaires et synonymes.

8.1 Diagramme de Classes

Voir **Annexe A** pour le code PlantUML complet du diagramme de classes.

8.2 Modèle Logique de Données (MLD)

8.2.1 Table `indexes_registry`

Colonne	Type	Contrainte	Description
`id`	UUID	PK	Identifiant
`name`	VARCHAR(100)	NOT NULL	Nom de l'index
`platform_id`	UUID	NOT NULL	Plateforme propriétaire (FK logique → A
`es_alias`	VARCHAR(150)	NOT NULL, UNIQUE	Alias Elasticsearch actif (ex : `products_l`
`doc_count`	BIGINT	NOT NULL, DEFAULT 0	Nombre de documents indexés
`status`	VARCHAR(20)	NOT NULL	`active` / `rebuilding` / `deleted`
`created_at`	TIMESTAMPTZ	NOT NULL	Date de création
`updated_at`	TIMESTAMPTZ	NOT NULL	Dernière mise à jour

Attention

Contrainte UNIQUE (v1.1) : `UNIQUE(name, platform_id)` — deux plateformes différentes peuvent avoir un index nommé `"products"`. L'unicité est scopée par plateforme.

8.2.2 Table `index_schemas`

Colonne	Type	Contrainte	Description
`id`	UUID	PK	Identifiant
`index_id`	UUID	FK → `indexes_registry.id`	Index parent
`field_name`	VARCHAR(100)	NOT NULL	Nom du champ
`field_type`	VARCHAR(20)	NOT NULL	`text` / `keyword` / `number` / `date` / `boolean`
`searchable`	BOOLEAN	NOT NULL	Inclus dans la recherche full-text
`filterable`	BOOLEAN	NOT NULL	Utilisable comme filtre
`sortable`	BOOLEAN	NOT NULL	Utilisable comme critère de tri
`autocomplete`	BOOLEAN	NOT NULL	Utilisé pour l'auto-complétion
`boost_weight`	INTEGER	NULL, DEFAULT 1	Poids dans le score
`created_at`	TIMESTAMPTZ	NOT NULL	Date de création
`updated_at`	TIMESTAMPTZ	NOT NULL	Dernière mise à jour

8.2.3 Table `search_history`

Colonne	Type	Contrainte	Description
`id`	BIGSERIAL	PK	Identifiant auto-incrémenté
`user_id`	UUID	NULL	Utilisateur (`users_auth.id` = `sub` du JWT). NULL si anonyme
`platform_id`	UUID	NOT NULL	Plateforme (FK logique → Auth `platforms.id`)
`index_name`	VARCHAR(100)	NOT NULL	Index interrogé
`query`	TEXT	NOT NULL	Requête saisie
`filters_applied`	JSONB	NULL	Filtres utilisés (v1.1)
`result_count`	INTEGER	NOT NULL	Nombre de résultats retournés
`took_ms`	INTEGER	NOT NULL	Temps de réponse en ms
`created_at`	TIMESTAMPTZ	NOT NULL	Horodatage de la recherche

8.2.4 Table `search_configs`

Colonne	Type	Contrainte	Description
`index_id`	UUID	PK, FK → `indexes_registry.id`	Index concerné
`analyzer`	VARCHAR(50)	NOT NULL	Analyseur linguistique (`french`, `english`)
`fuzzy_enabled`	BOOLEAN	NOT NULL, DEFAULT true	Fuzzy matching activé
`fuzzy_distance`	INTEGER	NOT NULL, DEFAULT 1	Distance d'édition (1 ou 2)
`highlight_enabled`	BOOLEAN	NOT NULL, DEFAULT true	Highlight dans les résultats (v1.1)
`min_score`	FLOAT	NULL	Seuil de pertinence minimum (v1.1)
`max_results`	INTEGER	NOT NULL, DEFAULT 100	Limite max de résultats (v1.1)
`updated_at`	TIMESTAMPTZ	NOT NULL	Dernière mise à jour

8.2.5 Table `synonyms`

Colonne	Type	Contrainte	Description
`id`	UUID	PK	Identifiant
`index_id`	UUID	FK → `indexes_registry.id`	Index parent
`term`	VARCHAR(100)	NOT NULL	Terme source
`equivalents`	TEXT	NOT NULL	Liste de synonymes (séparés par virgule)
`created_at`	TIMESTAMPTZ	NOT NULL	Date de création

Note

Renommage v1.1 : la colonne `synonyms` de la v1.0 est renommée `equivalents` pour cohérence avec le body API et éviter la confusion table/colonne.

8.2.6 Table `popular_searches` (v1.1 — documentée)

Colonne	Type	Contrainte	Description
`id`	UUID	PK	Identifiant
`index_name`	VARCHAR(100)	NOT NULL	Index concerné
`platform_id`	UUID	NOT NULL	Plateforme (FK logique → Auth)
`term`	VARCHAR(255)	NOT NULL	Terme recherché (anonymisé)
`search_count`	BIGINT	NOT NULL, DEFAULT 0	Nombre total de recherches
`last_searched_at`	TIMESTAMPTZ	NOT NULL	Dernière occurrence
`updated_at`	TIMESTAMPTZ	NOT NULL	Dernière agrégation

Contrainte : `UNIQUE(index_name, platform_id, term)`.

Cette table est alimentée par un job Celery périodique qui agrège les données de `search_history` de manière anonymisée (sans `user_id`). Elle sert à enrichir les suggestions d'auto-complétion et à alimenter l'endpoint `GET /api/v1/search/popular`.

9 Contrats d'API

Conventions API

Base URL : `/api/v1`. Tous les endpoints retournent du JSON. Tous les endpoints protégés requièrent le header `Authorization: Bearer <token>`, validé via `GET /auth/verify-token` (Auth v2.1).

Isolation multi-tenant (v1.1) : le `platform_id` est extrait du JWT vérifié. Toute opération est automatiquement scopée à la plateforme du token. Un index appartenant à une plateforme A n'est jamais accessible via un token de plateforme B.

Identité utilisateur (v1.1) : `user_id` = `sub` du JWT = `users_auth.id`.

9.1 Matrice d'autorisation par endpoint (v1.2)

Endpoint	Méthode	Public	JWT	S2S	Admin
`/search/health`	GET	✓			
`/search/indexes`	POST		✓	✓	
`/search/indexes`	GET		✓	✓	
`/search/indexes/{n}`	GET		✓	✓	
`/search/indexes/{n}/schema`	PUT		✓	✓	
`/search/indexes/{n}`	DELETE		✓	✓	
`/search/indexes/{n}/reindex`	POST		✓	✓	
`/search/indexes/{n}/documents`	POST		✓	✓	
`/search/indexes/{n}/documents/{d}`	PUT		✓	✓	
`/search/indexes/{n}/documents/{d}`	DELETE		✓	✓	
`/search/indexes/{n}/documents/bulk`	POST		✓	✓	
`/search/query`	POST		✓	✓	
`/search/autocomplete`	GET		✓	✓	
`/search/history`	GET		✓		
`/search/history`	DELETE		✓		
`/search/history/by-user/{id}`	DELETE			✓	
`/search/popular`	GET		✓	✓	
`/search/no-results`	GET			✓	✓
`/search/indexes/{n}/config`	GET		✓	✓	
`/search/indexes/{n}/config`	PUT		✓	✓	
`/search/indexes/{n}/synonyms`	PUT		✓	✓	
`/search/stats`	GET			✓	✓
`/search/stats/terms`	GET			✓	✓

Note

Légende :

- **Public** : aucune authentification requise.
- **JWT** : token Bearer utilisateur, validé via `GET /auth/verify-token`. L'accès est scopé automatiquement à la plateforme du token.
- **S2S** : token Service-to-Service, validé via `POST /auth/s2s/introspect`. Utilisé pour les appels machine-to-machine (indexation automatisée, purge RGPD, stats).
- **Admin** : réservé aux opérateurs AG Technologies. Authentification S2S + vérification d'un rôle admin global (vérifié via Service Users).

Règle v1.2 : les endpoints `history` (GET, DELETE) sont strictement personnels — un utilisateur ne voit et ne supprime que son propre historique. La purge RGPD `by-user/{id}` est S2S-only (orchestrée par Users). Les endpoints `stats` et `no-results` sont réservés aux tokens S2S ou admin car ils exposent des données agrégées cross-utilisateurs.

9.2 Health Check

9.2.1 `GET /api/v1/search/health`

Vérifie l'état du service et de ses dépendances. Aucune authentification requise.

Listing 2 – Réponse 200 — Health Check

```
{
  "status": "healthy",
  "database": "ok",
  "redis": "ok",
  "elasticsearch": "ok",
  "version": "1.1.0"
```

```
}
```

Codes : 200 OK, 503 Service Unavailable (si une dépendance est down).

9.3 Gestion des Index

9.3.1 `POST /api/v1/search/indexes`

Création d'un nouvel index. Le `platform_id` est extrait automatiquement du JWT.

Listing 3 – Body POST /api/v1/search/indexes

```
{
  "name": "products",
  "analyzer": "french",
  "fields": {
    "title": { "type": "text", "searchable": true,
      "autocomplete": true, "boost_weight": 3 },
    "price": { "type": "number", "filterable": true,
      "sortable": true },
    "category": { "type": "keyword", "filterable": true }
  }
}
```

Codes : 201 Created, 400 Validation error, 401 Unauthorized, 409 Index already exists (pour cette plateforme).

9.3.2 `GET /api/v1/search/indexes`

Liste des index de la plateforme authentifiée. Codes : 200, 401.

9.3.3 `GET /api/v1/search/indexes/{indexName}`

Détail d'un index (schéma, config, stats). Codes : 200, 401, 403 (index d'une autre plateforme), 404.

9.3.4 `PUT /api/v1/search/indexes/{indexName}/schema`

Ajout de nouveaux champs au schéma existant. Codes : 200, 400, 401, 403, 404.

Attention

Seul l'ajout de champs est supporté. La modification du type d'un champ existant nécessite une réindexation complète via `POST .../reindex`.

9.3.5 `DELETE /api/v1/search/indexes/{indexName}`

Suppression de l'index et de tous ses documents. Action irréversible. Codes : 204, 401, 403, 404.

9.3.6 `POST /api/v1/search/indexes/{indexName}/reindex`

Reconstruction complète de l'index via la stratégie alias Elasticsearch : création d'un nouvel index physique, migration des données, bascule atomique de l'alias, suppression de l'ancien index. Zéro downtime.

Codes : 202 Accepted (tâche Celery lancée), 401, 403, 404.

Listing 4 – Réponse 202 — Reindex

```
{
  "message": "Reindex started",
  "task_id": "celery-task-uuid",
  "index": "products",
  "status": "rebuilding"
}
```

9.4 Indexation

9.4.1 `POST /api/v1/search/indexes/{indexName}/documents`

Indexation d'un document (ajout ou mise à jour).

Listing 5 – Body POST document

```
{
  "doc_id": "prod-123",
  "data": {
    "title": "Nike Air Max 270",
    "price": 35000,
    "category": "Chaussures",
    "is_boosted": false,
    "created_at": "2026-01-10T08:00:00Z"
  }
}
```

Codes : 201 Created / 200 Updated, 400 Validation error, 401, 403, 404 (index inexistant).

9.4.2 `PUT /api/v1/search/indexes/{indexName}/documents/{docId}`

Mise à jour complète d'un document existant. Codes : 200, 400, 401, 403, 404.

9.4.3 `DELETE /api/v1/search/indexes/{indexName}/documents/{docId}`

Suppression d'un document de l'index. Codes : 204, 401, 403, 404.

9.4.4 `POST /api/v1/search/indexes/{indexName}/documents/bulk`

Indexation en masse.

Listing 6 – Body POST bulk

```
{
  "operations": [
    { "action": "index", "doc_id": "prod-001",
      "data": { "title": "Produit A", "price": 5000 } },
    { "action": "index", "doc_id": "prod-002",
      "data": { "title": "Produit B", "price": 8000 } },
    { "action": "delete", "doc_id": "prod-999" }
  ]
}
```

Réponse 207 Multi-Status : résumé avec succès et erreurs par opération.

Listing 7 – Réponse 207 — Bulk

```
{
  "total": 3,
  "succeeded": 2,
  "failed": 1,
  "errors": [
    { "doc_id": "prod-999", "action": "delete",
      "error": "Document not found" }
  ]
}
```

Codes : 207 Multi-Status, 400, 401, 403, 404.

9.5 Recherche

9.5.1 `POST /api/v1/search/query`

Recherche full-text principale.

Listing 8 – Body POST /api/v1/search/query

```
{
  "index": "products",
  "query": "chaussures nike",
  "search_type": "fulltext",
  "filters": [
    { "field": "price", "operator": "range",
      "min": 5000, "max": 50000 },
    { "field": "category", "operator": "eq",
      "value": "Chaussures" }
  ],
  "sort": { "field": "rating", "order": "desc" },
  "page": 1,
  "limit": 20,
  "include_facets": true
}
```

Listing 9 – Réponse 200 — Search

```
{
  "results": [
    { "doc_id": "prod-123", "score": 4.82,
      "data": { "title": "Nike Air Max 270",
                "price": 35000 },
      "highlights": { "title": "<em>Nike</em> Air Max 270" }
    }
  ],
  "total": 48,
  "page": 1,
  "limit": 20,
  "took_ms": 34,
  "facets": {
    "category": { "Chaussures": 28, "Accessoires": 12 },
    "price_range": { "0-10000": 10, "10000-50000": 30 }
  }
}
```

Codes : 200 OK, 400 Validation error (query vide, limit > 100, wildcard interdit), 401, 403, 408 Timeout (query ES > 5s), 429 Rate limited.

Note

v1.1 : le `limit` est plafonné à 100 (cf. section 6.2.1). Les wildcards en début de terme sont rejetés avec une erreur 400.

9.5.2 `GET /api/v1/search/autocomplete`

Auto-complétion en temps réel.
Query params : `index` (obligatoire), `prefix` (obligatoire), `limit` (défaut : 8, max : 20).

Listing 10 – Réponse GET /api/v1/search/autocomplete

```
{
  "suggestions": [
    { "text": "chaussures",
      "highlight": "<em>chau</em>ssures" },
    { "text": "chaussures de sport",
      "highlight": "<em>chau</em>ssures de sport" },
    { "text": "chaussettes",
      "highlight": "<em>chau</em>ssettes" }
  ],
  "took_ms": 12
}
```

Codes : 200, 400 (prefix manquant), 401, 403, 429.

9.6 Historique de Recherche

9.6.1 `GET /api/v1/search/history`

Historique de l'utilisateur authentifié. Query params : `page`, `limit`. Codes : 200, 401.

9.6.2 `DELETE /api/v1/search/history`

Suppression de tout l'historique de l'utilisateur (RGPD). Codes : 204, 401.

9.6.3 `DELETE /api/v1/search/history/by-user/{userId}` (v1.1 — S2S)

Purge RGPD de l'historique d'un utilisateur. **Token S2S uniquement.** `userId` = `users_auth.id`.
Supprime toutes les entrées de `search_history` où `user_id = userId`.

Listing 11 – Réponse 200 — RGPD purge

```
{
  "message": "Search history purged",
  "user_id": "uuid",
  "entries_deleted": 342
}
```

Codes : 200, 401, 404, 500.

9.6.4 `GET /api/v1/search/popular`

Recherches populaires agrégées et anonymisées. Query params : `index` (obligatoire), `limit` (défaut : 10). Codes : 200, 400, 401.

9.6.5 `GET /api/v1/search/no-results`

Recherches sans résultats (endpoint admin / S2S). Query params : `index`, `from`, `to`. Codes : 200, 400, 401, 403.

9.7 Configuration

9.7.1 `GET /api/v1/search/indexes/{indexName}/config`

Récupère la configuration d'un index. Codes : 200, 401, 403, 404.

9.7.2 `PUT /api/v1/search/indexes/{indexName}/config`

Listing 12 – Body PUT config

```
{
  "analyzer": "french",
  "fuzzy_enabled": true,
  "fuzzy_distance": 1,
  "highlight_enabled": true,
  "min_score": 0.5,
  "max_results": 100
}
```

Codes : 200, 400, 401, 403, 404.

9.7.3 `PUT /api/v1/search/indexes/{indexName}/synonyms`

Listing 13 – Body PUT synonyms

```
{
  "synonyms": [
    { "term": "telephone",
      "equivalents": ["smartphone", "mobile", "cellulaire"] },
    { "term": "chaussure",
      "equivalents": ["basket", "sneaker", "shoe"] }
  ]
}
```

Codes : 200, 400, 401, 403, 404.

9.8 Statistiques

9.8.1 `GET /api/v1/search/stats`

Listing 14 – Réponse GET /api/v1/search/stats

```
{
  "total_searches": 125430,
  "avg_response_ms": 87,
  "by_index": {
    "products": { "searches": 98000, "avg_ms": 72 },
    "faq": { "searches": 27430, "avg_ms": 120 }
  }
}
```

Codes : 200, 401.

9.8.2 `GET /api/v1/search/stats/terms`

Termes les plus recherchés. Query params : `index` (obligatoire), `limit`, `from`, `to`. Codes : 200, 400, 401.

10 Contrats Inter-Services

10.1 Recherche → Auth (JWT utilisateur)

Paramètre	Valeur
Endpoint	`GET /auth/verify-token`
Auth	`Authorization: Bearer <access_token>`
Données extraites	`user_id` (`sub`), `platform_id`
Cache	Redis TTL 30s
Comportement en cas d'échec	Retourner 401 Unauthorized

10.2 Recherche → Auth (token S2S)

Paramètre	Valeur
Endpoint	`POST /auth/s2s/introspect`
Données	`client_id` (= `platform_id`), `service_name`
Usage	Appels S2S entrants (ex : indexation automatisée depuis AGT-Market)

10.3 Users → Recherche (Purge RGPD)

Paramètre	Valeur
Endpoint	`DELETE /api/v1/search/history/by-user/{userId}`
Auth	Token S2S
`userId`	`users_auth.id` (convention écosystème)
Séquence	Appelé par Users dans son workflow de purge RGPD, **avant** l'appel à Auth purge et Média purge
Données supprimées	Toutes les entrées `search_history` où `user_id = userId`
Tolérance erreurs	Si Recherche échoue (500), Users conserve le statut `deletion_in_progress` et peut retenter. Un cron de rattrapage peut relancer les purges échouées.

Note

Séquence RGPD complète (orchestrée par Users) :

1. Users marque le profil `status=deletion_in_progress`
2. Users appelle **Recherche** purge (`DELETE .../by-user/{userId}`)
3. Users appelle **Média** purge (`DELETE /api/v1/media/by-user/{userId}`)
4. Users appelle **Notification** purge (préférences + historique)
5. Users appelle **Auth** purge (`DELETE /auth/admin/purge/{authUserId}`)
6. Users finalise son propre hard delete

10.4 Contrat d'événements d'indexation

Les plateformes consommatrices publient des événements d'indexation sur RabbitMQ. Le Service Recherche consomme ces événements via des workers Celery.

10.4.1 Format de message standard

Listing 15 – Format événement d'indexation

```
{
  "event_type": "INDEX | UPDATE | DELETE",
  "platform_id": "uuid-platform",
  "index_name": "products",
  "doc_id": "prod-123",
  "payload": {
    "title": "Nike Air Max 270",
    "price": 35000
  },
  "version": 1,
  "timestamp": "2026-03-21T10:30:00Z",
  "idempotency_key": "uuid-unique"
}
```

10.4.2 Règles de traitement

Règle	Description
Idempotence	Chaque message contient un `idempotency_key` (UUID unique). Les doublons sont ignorés (vérification Redis, TTL 24h).
Retry	3 tentatives avec backoff exponentiel (1s, 5s, 25s)
DLQ (Dead Letter Queue)	Après 3 échecs, le message est routé vers une DLQ pour investigation manuelle
Versioning	Le champ `version` permet de gérer l'ordre des mises à jour concurrentes (last-write-wins basé sur version)
Routing key	`search.index.{platform_id}` — permet un filtrage par plateforme

11 Infrastructure

11.1 Composants d'Infrastructure

Composant	Phase	Détails
Elasticsearch 8.x	MVP	Moteur de recherche, auto-hébergé dans Docker
PostgreSQL 15+	MVP	Métadonnées, historique, synonymes, configs
Redis 7+	MVP	Cache auto-complétion, recherches populaires, rate limiting, idempotency keys
RabbitMQ	MVP	File d'attente pour indexation asynchrone (Celery)
Celery	MVP	Workers de consommation des événements d'indexation
Celery Beat	MVP	Jobs planifiés (agrégation popular_searches, purge)
Docker	MVP	Conteneurisation de l'API et d'Elasticsearch
SSD (stockage)	MVP	Recommandé pour les I/O Elasticsearch
API Embeddings	Futur	Recherche sémantique vectorielle

Attention

Ressources Elasticsearch : le moteur nécessite un minimum de 2 GB de RAM dédiés pour la JVM heap. Prévoir au moins 4 GB de RAM totale sur le serveur hébergeant Elasticsearch en production.

11.2 Variables d'Environnement

Variable	Description
`ELASTICSEARCH_URL`	URL du cluster Elasticsearch
`ELASTICSEARCH_API_KEY`	Clé d'API Elasticsearch
`DATABASE_URL`	URL de connexion PostgreSQL
`REDIS_URL`	URL de connexion Redis
`AUTH_SERVICE_URL`	URL du Service Auth
`BROKER_URL`	URL RabbitMQ (`amqp://...`)
`CELERY_RESULT_BACKEND`	Backend de résultats Celery (Redis)
`AUTOCOMPLETE_CACHE_TTL`	TTL cache auto-complétion (secondes)
`POPULAR_CACHE_TTL`	TTL cache recherches populaires (secondes)
`ES_HEAP_SIZE`	Taille heap JVM Elasticsearch (ex : 2g)
`ES_QUERY_TIMEOUT`	Timeout requête ES (défaut : 5s)
`SECRET_KEY`	Clé secrète Django
`RATE_LIMIT_SEARCH`	Limite recherche/min/plateforme (défaut : 600)
`RATE_LIMIT_INDEX`	Limite indexation/min/plateforme (défaut : 120)
`MAX_INDEXES_PER_PLATFORM`	Quota max d'index actifs par plateforme (défaut : 20)
`MAX_BULK_OPERATIONS`	Max opérations par requête bulk (défaut : 500)
`MAX_BULK_SIZE_MB`	Taille max requête bulk en MB (défaut : 10)
`MAX_DOC_SIZE_MB`	Taille max d'un document en MB (défaut : 1)
`MAX_FIELDS_PER_INDEX`	Max champs par schéma d'index (défaut : 100)

11.3 Docker Compose (Extrait)

Listing 16 – docker-compose.yml — Search Service

```yaml
services:
  search-service:
    build: ./search-service
    ports:
      - "3004:3004"
    environment:
      - ELASTICSEARCH_URL=http://elasticsearch:9200
      - DATABASE_URL=postgresql://...
      - REDIS_URL=redis://redis:6379
      - AUTH_SERVICE_URL=http://auth-service:3001
      - BROKER_URL=amqp://guest:guest@rabbitmq:5672//
    depends_on:
      - elasticsearch
      - postgres
      - redis
      - rabbitmq

  celery-worker:
    build: ./search-service
    command: celery -A search worker -l info -Q indexation
    environment:
      - BROKER_URL=amqp://guest:guest@rabbitmq:5672//
      - DATABASE_URL=postgresql://...
      - ELASTICSEARCH_URL=http://elasticsearch:9200
      - REDIS_URL=redis://redis:6379
    depends_on:
      - rabbitmq
      - elasticsearch

  celery-beat:
    build: ./search-service
    command: celery -A search beat -l info
    depends_on:
      - rabbitmq

  elasticsearch:
    image: elasticsearch:8.13.0
    environment:
      - discovery.type=single-node
      - ES_JAVA_OPTS=-Xms2g -Xmx2g
      - xpack.security.enabled=false
    volumes:
      - es_data:/usr/share/elasticsearch/data
    ports:
      - "9200:9200"

  postgres:
    image: postgres:16-alpine
    volumes:
      - search_pg_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine

  rabbitmq:
    image: rabbitmq:3-management-alpine
```

```
    ports:
      - "5672:5672"
      - "15672:15672"

volumes:
  es_data:
  search_pg_data:
```

12 Recommandations & Évolutions

12.1 Réindexation sans Downtime (Alias Elasticsearch)

La stratégie recommandée utilise les alias Elasticsearch. L'index actif est référencé par un alias (ex : `products_live`). Une réindexation crée un nouvel index `products_v2`, le peuple entièrement, puis bascule l'alias en une opération atomique. L'ancien index est supprimé après validation.

Cette stratégie est implémentée par l'endpoint `POST /api/v1/search/indexes/{indexName}/reindex` et gérée en tâche de fond par Celery.

12.2 Recherche Sémantique par Embeddings

Pour la phase future, chaque document est enrichi à l'indexation d'un embedding calculé via l'API OpenAI, stocké dans un champ `dense_vector` Elasticsearch. Les requêtes sont également transformées en vecteurs, puis la similarité cosinus retourne les documents sémantiquement proches.

12.3 Synonymes Auto-Enrichis

Les termes recherchés sans résultat sont analysés périodiquement via un job Celery Beat qui identifie les termes récurrents dans `search_history` où `result_count = 0` et propose des synonymes aux admins de chaque plateforme.

12.4 Monitoring Elasticsearch

Surveiller en permanence : santé du cluster (green/yellow/red), taux de rejet (circuit breaker), latence des requêtes (P50, P95, P99), espace disque et mémoire heap JVM. Alertes à configurer dès que le heap dépasse 85% ou que le cluster passe en yellow.

12.5 Suppression : Stratégies

Stratégie	Comportement	Usage
Suppression document	Retrait de l'index ES immédiatement	Mise à jour depuis la plateforme source
Suppression index	Index ES supprimé + registre PostgreSQL mis à jour	Décision de la plateforme
Suppression historique	Suppression physique des lignes `search_history`	Demande utilisateur (RGPD)
Purge RGPD S2S	Suppression de tout l'historique d'un `user_id`	Orchestré par Users

A Annexe A — Diagramme de Classes (PlantUML)

Listing 17 – Code PlantUML — Diagramme de Classes Search Service v1.2

```
@startuml Search Service - Class Diagram v1.2

skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "indexes_registry" as idx {
  * id : UUID <<PK>>
  --
  * name : VARCHAR(100)
  * platform_id : UUID <<FK logique -> Auth platforms.id>>
  * es_alias : VARCHAR(150) <<UNIQUE>>
  doc_count : BIGINT = 0
  * status : VARCHAR(20)
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  UNIQUE(name, platform_id)
}

entity "index_schemas" as sch {
  * id : UUID <<PK>>
  --
  * index_id : UUID <<FK>>
  * field_name : VARCHAR(100)
  * field_type : VARCHAR(20)
  * searchable : BOOLEAN
  * filterable : BOOLEAN
  * sortable : BOOLEAN
  * autocomplete : BOOLEAN
  boost_weight : INTEGER = 1
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

entity "search_configs" as cfg {
  * index_id : UUID <<PK, FK>>
  --
  * analyzer : VARCHAR(50)
  * fuzzy_enabled : BOOLEAN = true
  * fuzzy_distance : INTEGER = 1
  * highlight_enabled : BOOLEAN = true
  min_score : FLOAT
  * max_results : INTEGER = 100
  * updated_at : TIMESTAMPTZ
}

entity "synonyms" as syn {
  * id : UUID <<PK>>
  --
  * index_id : UUID <<FK>>
  * term : VARCHAR(100)
  * equivalents : TEXT
  * created_at : TIMESTAMPTZ
}

entity "search_history" as hist {
  * id : BIGSERIAL <<PK>>
```

```
  --
  user_id : UUID
  * platform_id : UUID
  * index_name : VARCHAR(100)
  * query : TEXT
  filters_applied : JSONB
  * result_count : INTEGER
  * took_ms : INTEGER
  * created_at : TIMESTAMPTZ
}

entity "popular_searches" as pop {
  * id : UUID <<PK>>
  --
  * index_name : VARCHAR(100)
  * platform_id : UUID
  * term : VARCHAR(255)
  * search_count : BIGINT = 0
  * last_searched_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  UNIQUE(index_name, platform_id, term)
}

class "SearchQuery" as sq {
  +query : String
  +index : String
  +search_type : Enum
  +filters : List<Filter>
  +sort : SortOption
  +page : Integer
  +limit : Integer
  +include_facets : Boolean
  --
  +execute() : SearchResult
  +validate() : Boolean
  +buildESQuery() : JSON
}

class "SearchResult" as sr {
  +results : List<Hit>
  +total : Integer
  +page : Integer
  +limit : Integer
  +took_ms : Integer
  +facets : Map<String, Map>
  --
  +toResponse() : JSON
}

class "AutocompleteQuery" as aq {
  +index : String
  +prefix : String
  +limit : Integer
  --
  +execute() : List<Suggestion>
}

class "SearchDocument" as doc {
  +doc_id : String
  +index_name : String
```

```
  +platform_id : UUID
  +data : JSON
  +indexed_at : Timestamp
  --
  +index() : void
  +update() : void
  +delete() : void
}

idx ||--o{ sch : "has fields"
idx ||--|| cfg : "configured by"
idx ||--o{ syn : "has synonyms"
idx ||--o{ hist : "logged in"
sq --> idx : "targets"
sq --> sr : "produces"
doc --> idx : "belongs to"
aq --> idx : "queries"

@enduml
```

B Annexe B — Glossaire

Terme	Définition
Elasticsearch	Moteur de recherche distribué basé sur Apache Lucene
Index	Collection de documents structurés dans Elasticsearch
Mapping	Définition du schéma d'un index (types et propriétés des champs)
Full-text search	Recherche dans le contenu textuel avec analyse linguistique
Fuzzy matching	Tolérance aux fautes de frappe basée sur la distance d'édition
Facette	Compteur de résultats par valeur de filtre (ex : catégorie : 42)
Boost	Mécanisme d'élévation du score de certains documents
Stemming	Réduction d'un mot à sa racine (chaussures → chaussure)
Embedding	Représentation vectorielle d'un texte pour la recherche sémantique
Alias ES	Pointeur vers un index ES, permet la réindexation sans downtime
Shard	Unité de distribution des données dans Elasticsearch
TTL	Time To Live — durée de vie d'une entrée en cache Redis
DLQ	Dead Letter Queue — file d'attente pour messages en échec
S2S	Service-to-Service — communication inter-microservices
Celery	Framework Python de tâches asynchrones distribuées

AG Technologies — Document confidentiel à usage interne exclusivement.
Search Service CDC v1.2 — 21 mars 2026